ENSAR at AraSeg Shared Task 2026: An Ensemble of Encoders With Agentic Error Refinement

Noor Emam **Omar Saqr** **Mostafa Gafaar** **Ali Elaswad**
The American University in Cairo
{noortaytoy1, omar_saqr, mostafa21314, alyelaswad}@aucegypt.edu

Abstract

We describe ENSAR, our submission to the AraSeg 2026 shared task on Arabic sentence segmentation, where a system must decide, for every word in a document, whether a sentence ends there. The four tracks strip punctuation, paragraph breaks, both, or neither. We start from an ensemble of five fine-tuned encoders decoded with a length-aware semi-Markov program. It is strong, but most of its mistakes are places where all five models are wrong and all five assign high confidence, so averaging them or moving a threshold changes nothing. ENSAR adds a reasoning stage on top. Two LLM juries, each holding a written policy learned from its own graded errors on the training split, debate every document and apply only the edits both endorse. Our system ranked first on three of the four tracks in both the closed and open settings.

1 Introduction

This paper presents our participation in the AraSeg 2026 Shared Task on Arabic Sentence Segmentation (Elkholy et al., 2026b), organized as part of ArabicNLP 2026. The task is to decide, for every word in a document, whether a sentence ends there. The four tracks strip punctuation, paragraph breaks, both, or neither. Read et al. (2012) showed that systems which look only at full stops and question marks score well on clean newspaper text and drop sharply on anything else. The problem is sharper in Arabic, where punctuation is used inconsistently, a mark that ends a sentence in one text separates clauses in another, and some genres carry no reliable marks at all (Elkholy et al., 2026a). The conjunction *wa* ('and') can start a new sentence or link two clauses inside one, and nothing on the page tells you which (Farghaly and Shaalan, 2009). In the no-punctuation tracks there is no mark to look at at all. The design follows from a measurement (Appendix B): nearly half of our ensemble's development errors are sites where all five voters are wrong and all five are confident. Moving a threshold, recalibrating, or adding a voter cannot fix these, because no voter disagrees with the mistake in the first place. Our system therefore pairs the ensemble with a reasoning stage that learns each genre's annotation convention explicitly (Figures 1 and 2, Appendix A): five fine-tuned encoders propose boundaries through a length-aware semi-Markov decode, and two LLM juries (Verga et al., 2024), sharing a model but differing in their learned policies written from their own graded errors on the training split, work out what kind of text each document is and apply only unanimous edits. We have released our code, the full prompts, and all six learned policies.[1] Our main contributions are (i) an encoder ensemble whose semi-Markov decode trades boundary probability against segment-length plausibility, first on three of four tracks (Section 3); (ii) a training loop that improves a written policy instead of weights, graded by the annotator's labels (Section 3, Figure 1); (iii) an ablation separating the learned conventions from the model (Section 5); and (iv) the policy files themselves (Section 5, Appendix C).

2 Background

AraSeg task setup. AraSeg 2026 (Elkholy et al., 2026b) frames Arabic sentence segmentation as binary token classification: given a document as a sequence of whitespace tokens $w_1, \ldots, w_n$, a system predicts $y_i \in \{0, 1\}$ for each token, where $y_i = 1$ marks a sentence boundary after w_i. Punctuation marks are separate tokens; the boundary label falls on the sentence-final punctuation when present, and otherwise on the final word. The task defines four tracks crossing the availability of punctuation and paragraph structure: **PA** (both), **NP** (punctuation only), **NoPnx-PA** (paragraphs only), and **NoPnx-NP** (neither), where paragraph

[1] https://github.com/noortaytoy1/araseg-competition

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 944–955
October 28-29, 2026 ©2026 Association for Computational Linguistics

breaks appear as explicit newline tokens and the no-punctuation tracks are produced by deleting punctuation from the same documents, so their boundaries carry no surface marker. The corpus spans eight genres with 174 training, 222 development, and 262 test documents per track (Elkholy et al., 2026a), and systems are scored by macro-averaged per-document boundary F_1. Both a closed track (provided data only) and an open track (external resources allowed) are offered; we participate in all four tracks under both.

Sentence segmentation methods. Neural work removes the reliance on punctuation by casting segmentation as supervised token classification over a pretrained encoder: the self-supervised WtP (Minixhofer et al., 2023) and its successor SaT (Frohmann et al., 2024) train punctuation-agnostic transformer encoders that segment robustly across many languages. These systems are accurate but opaque, they emit a probability per token and cannot say *why* a boundary was placed or which convention a given document follows, which is precisely where genre-dependent Arabic segmentation is hardest. A separate line of work lets a frozen LLM improve at a task by writing in natural language rather than updating weights: Reflexion turns task feedback into verbal self-corrections (Shinn et al., 2023), Self-Refine iterates a model against its own critique (Madaan et al., 2023), and ExpeL distils reusable insights from an agent's own successes and failures (Zhao et al., 2024). We adapt this idea to segmentation, but ground it in token-level gold: rather than optimising a prompt from aggregate scores, each of our juries induces a persistent, genre-aware segmentation policy from its own graded errors on the training split (Section 3).

3 System Overview

ENSAR has two stages: an encoder ensemble that proposes a boundary draft, and a reasoning stage of two LLM juries that edit it.

Encoder ensemble. We fine-tune four pretrained encoders, AraBERTv0.2 base and large (Antoun et al., 2020), Arabic-BERT-large (Safaya et al., 2020), and XLM-RoBERTa-large (Conneau et al., 2020), into five voters, with two of the large AraBERT twins adding FGM adversarial training (Miyato et al., 2017) and an auxiliary supervised-contrastive loss (Khosla et al., 2020) for diversity. Each voter is a token classifier over overlapping

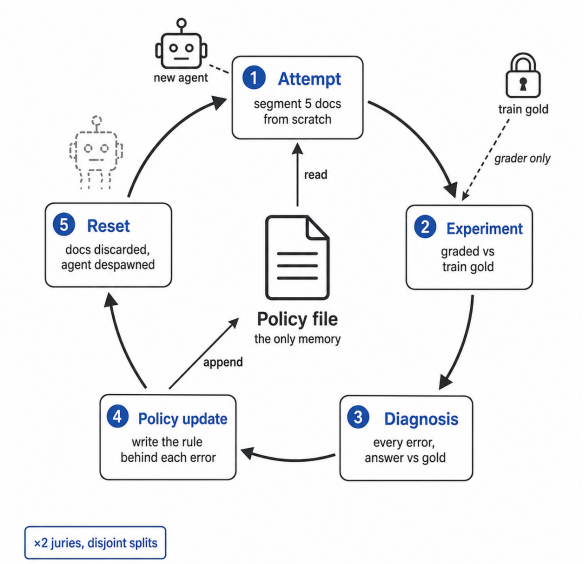


Figure 1: The jury training loop (Section 3). In each batch the jury reads its policy file, segments five training documents from scratch, is graded against the training gold, sees every error beside the annotator's answer, and writes the rule behind each error into the file. The documents are then discarded and the agent is periodically replaced by a fresh one that receives only the policy file, which is the sole memory carried across batches; the gold is visible to the grader only. Two juries are trained this way for each track pair.

word windows; per-token boundary probabilities are averaged across voters and windows. Rather than thresholding each token independently, we decode the averaged probabilities with a semi-Markov dynamic program (Sarawagi and Cohen, 2004) that scores a boundary set B against a segment-length prior fit on the training split:

$$\hat{B} = \arg\max_{B} \sum_{i \in B} \log p_i + \lambda \sum_{s \in \mathrm{seg}(B)} \mathcal{L}(|s|) - \beta |B|, \quad (1)$$

where B is a candidate set of boundary positions, p_i is the averaged probability that a boundary follows token i, $\mathrm{seg}(B)$ is the set of segments that B induces, $\mathcal{L}(|s|) = \log \hat{P}(|s|)$ is the log-probability of a segment of length $|s|$ under the training-split length prior, and $\lambda, \beta \geq 0$ weight the length prior and a per-boundary penalty. A boundary is thus kept only when its probability and the segment lengths it creates are jointly more plausible than leaving it out, the length term supplies the structure that punctuation would otherwise give in the NoPnx tracks. The prior is then refit on the first-pass segments and the document is decoded once more, yielding a two-pass, document-adaptive variant.

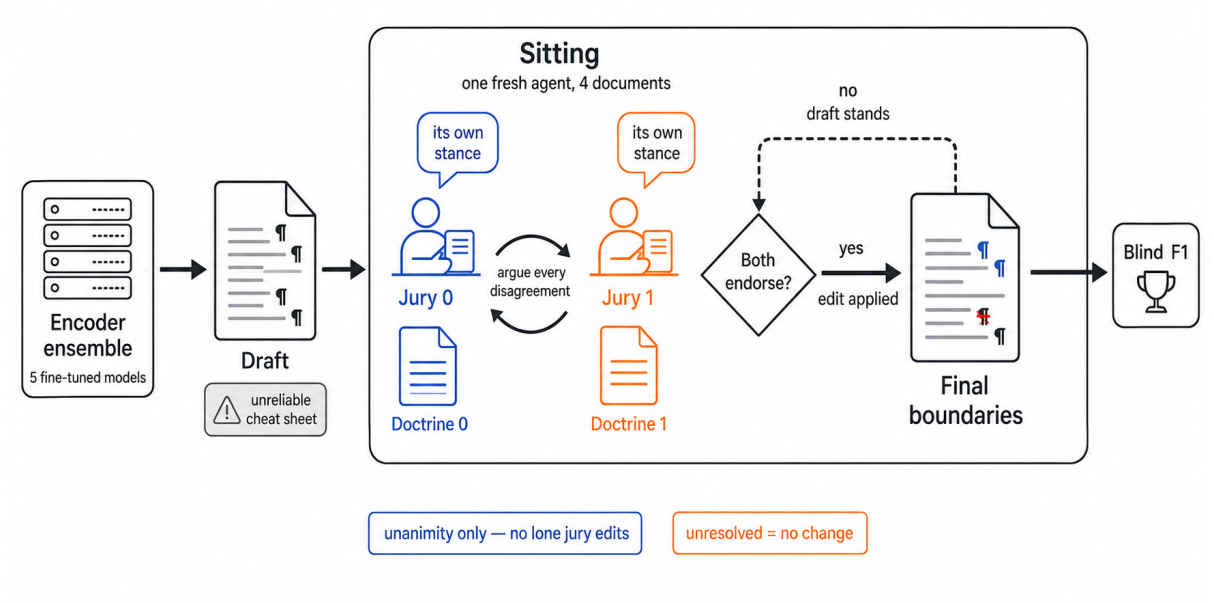


Figure 2: Adjudication at inference (Section 3). The encoder ensemble (five fine-tuned voters; a tuned three-voter subset for the Pnx-PA submission) produces a draft segmentation, which enters a sitting: one fresh agent holding four documents and both learned doctrines, treating the draft as an unreliable cheat sheet. Jury 0 and Jury 1 each state a stance from their own doctrine and argue every disagreement; an edit is applied only when both endorse it, and an unresolved site leaves the draft unchanged. The final boundaries are scored with the organizers' script.

This decoded draft is what the juries receive. Hyperparameters, per-voter recipes, and the tuned per-track configurations, including a three-voter subset for PA, are given in Section 4 and Appendix A.

Learning a policy from graded errors. A jury is defined entirely by one text file it writes itself: a policy of genre-specific segmentation rules, each rule stated together with the graded error that produced it. Training runs in batches of five documents through five steps (Figure 1). **(1) Attempt:** the jury reads its current policy and segments each document from scratch. **(2) Experiment:** the attempt is scored against training gold, boundary by boundary. **(3) Diagnosis:** every false cut and miss is returned in context, the jury's answer beside the annotator's at the same position. **(4) Policy update:** the jury writes the rule behind each error into its file, using its own failure as the worked example; contradicted rules are struck through in place, so the record shows what was tried and why it was abandoned. **(5) Reset:** the documents are discarded and the next batch starts from the updated policy. We periodically retire the agent and respawn a fresh one holding only the policy file, since one agent carrying its full context across every batch is costly and more prone to hallucination. Grading the boundaries rather than the weights is what makes the learned object a readable, auditable policy instead of a model update. We train one jury pair per track family.

Adjudication: only edits both juries endorse. At inference, two juries, the same model under two independently trained policies, read one document at a time (Figure 2). The ensemble draft is presented in-text as marked boundaries and described as an unreliable cheat sheet: each jury may consult it but argues its own reading from its policy, and the juries produce a list of *edits* to the draft (boundaries to add or remove) rather than re-segmenting from scratch. They argue every site on which they differ, with a rule earned from a graded error in the matching register outranking unsupported intuition. Only edits *both* juries endorse are applied; unresolved disagreements leave the draft unchanged, and an unchanged document is an admissible verdict. Edits are applied strictly, an edit lands only where its stated index still holds its stated token, which keeps the whole stage auditable. Training and inference deliberately differ: a jury trains by segmenting from scratch, never by correcting drafts, so it cannot learn to defer to them; the learned policy of where sentences end then applies to both writing and judging, and Table 1 shows that it transfers.

4 Experimental Setup

Data. The encoders are trained on the training split and selected using the development split, while the juries learn exclusively from the training documents and their gold labels. The released test split is used only for the controlled comparison reported in Table 1. We participate in all four tracks under both the closed and open settings.

Preprocessing. We apply no normalization, alef and ya variants, ta marbuta, diacritics, and tatweel are all preserved because the whitespace-token sequence defines the label space. We also use no external segmenters. Each encoder applies its own subword tokenizer to the provided tokens, with the word-level boundary label assigned to the final subword.

Models and evaluation. The five encoder voters are fine-tuned using Hugging Face Transformers, while the juries run Claude Opus 5 at maximum reasoning effort. The official evaluation metric is macro-averaged, per-document boundary F_1, computed using the organizers' evaluation script. Full hyperparameters, per-voter training recipes, decoding settings, software versions, and hardware specifications are provided in Appendix A.

5 Results and Analysis

Official results. ENSAR placed first on the official leaderboards for Pnx-PA, Pnx-NP, and NoPnx-NP in both the closed and open settings, achieving blind F_1 scores of 95.30, 92.0, and 89.1, respectively, and reached 90.0 on NoPnx-PA. Because the leaderboard results cannot separate the contribution of the encoder stage from that of the reasoning stage, we evaluate the reasoning stage directly on the released test split in Table 1.

Ablation setup. All variants use the same documents, gold labels, ensemble drafts, prompts, unanimity rule, and model, differing only in the context provided to the juries. The **zero-shot** variant removes the doctrine: the juries are told that they are untrained and edit using only their existing knowledge of Arabic. The **doctrine** variant corresponds to the submitted reasoning stage. Without two distinct doctrines the zero-shot juries behave more like one policy, weakening the unanimity filter—an asymmetry to keep in mind below.

A policy, not a threshold. The most diagnostic quantities are the directions of the edits (Table 2). On NoPnx-NP the juries add far more boundaries than they remove; on NoPnx-PA they remove more than they add. The ensemble therefore under-segments when both punctuation and paragraph structure are absent, but over-segments when paragraph tokens are present, and the juries correct each case in the appropriate direction. These opposite patterns are inconsistent with a simple global confidence adjustment through a single threshold, temperature, or decoding penalty. Instead, the reasoning layer conditions its decisions on the type of document it receives. On the punctuated tracks, it makes far fewer edits than on the corresponding no-punctuation tracks (Table 2), and leaves more than half of the documents unchanged. Although the gains are smaller, they remain positive, indicating that the policy contributes most where punctuation is absent and remains conservative where punctuation already supplies structure.

Scaffolding versus policy. The zero-shot variant separates the doctrine gain into two components. On NoPnx-NP, of the total gain of $+4.85$, $+2.96$ comes from the model-and-debate scaffolding, while the remaining $+1.89$ comes from the learned policy. The doctrine's margin over the zero-shot variant is positive across all four tracks, ranging from $+1.89$ to $+3.88$, and is statistically significant in every case. On the punctuated tracks, where the ensemble drafts are already strong, the value of the policy appears most clearly in its restraint: the doctrine-guided juries make far fewer edits than the zero-shot juries (Table 2). Only the doctrine variant preserves or improves the score. The policy therefore teaches the juries both when to intervene and when to leave a correct draft unchanged, and the ordering of the variants indicates that this restraint is learned rather than inherent to the model or debate format.

Memorization. Because the NoPnx data are produced by deleting punctuation from public source documents, a jury could, in principle, recall the original punctuation marks rather than infer the segmentation convention. Such recall would reside in the model's pretrained weights rather than in the learned doctrine and should therefore also benefit the zero-shot variant. Instead, the doctrine variant outperforms the zero-shot variant on every track, indicating that memorization alone cannot account for the observed gains.

	NoPnx-NP	NoPnx-PA	NP	PA
Ensemble	84.64	87.05	92.83	94.56
+ juries, zero-shot	87.59	88.71	91.42	92.18
+ juries, doctrine	**89.49**	**92.59**	**93.61**	**94.92**
Doctrine vs. ensemble, paired over documents				
95% CI of Δ	$[3.6, 6.1]$	$[4.1, 7.0]$	$[0.4, 1.2]$	$[0.05, 0.67]$

Table 1: Ablation on the released test split using macro-averaged F_1. The variants differ only in the jury context: no learned doctrine (zero-shot) or the learned doctrine.

Doctrine juries, by direction			
	Add	**Remove**	**Total**
NoPnx-NP	923	417	1,340
NoPnx-PA	547	722	1,269
Total edits, doctrine vs. zero-shot juries			
	Doctrine	**Zero-shot**	
Pnx-NP	395	780	
Pnx-PA	254	760	

Table 2: Edits the juries apply to the ensemble draft on the released test split.

Genre	**Words per sentence**
Children's picture book	10.6
Children's magazine	14.3
Classical folk narrative	11
Translated monograph	20.3

Table 3: Approximate sentence-length expectations, by genre, recorded in the learned policies.

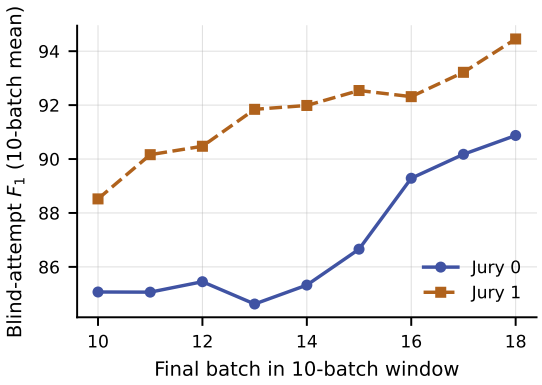


Figure 3: Blind-attempt F_1 of the two NoPnx-PA juries during training, averaged over a moving window of 10 batches (50 documents), in curriculum order. Each attempt is scored before the gold for its batch is revealed and no document is seen twice, so the curve reflects performance on unseen documents as the policy files grow. The juries train on disjoint halves of the corpus; NoPnx-PA is the track whose full training run is banked end to end, and the other tracks' attempts are not recoverable in curriculum order.

What the learned policies contain. Because the policies are stored as plain text, we can inspect what the system learned. One jury's first rule for any new document is to count the words between paragraph breaks and, when the lines are short, place boundaries at the breaks and almost nowhere else. The jury wrote this rule after its attempt on an exam-question bank was graded, having initially segmented the document according to prose-based assumptions. The policies also encode genre-specific sentence-length expectations (Table 3), each introduced after the jury found that treating these genres alike reduced accuracy. Appendix C presents a rule that was induced, refuted, and retired in writing, while Appendix D shows both juries applying their policies to complete documents. Late in training, both juries independently made "what kind of text is this?" their explicit first step. None of these rules is explicit in the encoder ensemble, and none was written by us. Figure 3 shows the two NoPnx-PA juries' blind-attempt F_1 rising as their policy files grow.

6 Conclusion

We presented ENSAR, an encoder ensemble refined by a reasoning layer (Figures 1 and 2) whose policy is learned from graded errors and stored as text. It placed first on three of four AraSeg 2026 tracks. A controlled ablation credits the second layer with close to five macro-F_1 points where punctuation is absent and under one where present. The remaining headroom is thus definitional: genre conventions recoverable from graded errors and statable in language; future work extends the loop to dialectal Arabic and to cheaper policy carriers.

Limitations

The reasoning layer is expensive: adjudicating one document costs on the order of tens of thousands of tokens, several orders of magnitude more than an encoder forward pass, so the method suits settings where segmentation quality outweighs throughput. The juries are a proprietary model accessed through an API, decoding is not deterministic, and exact reproduction depends on that model version remaining available; we release the learned policies and the prompts so the stage can be inspected even where it cannot be re-derived bit for bit. A second full run on 50 NoPnx-NP test documents differed from the first by 0.48 F_1; the effect is an order of magnitude larger than the run-to-run variability. When both juries share a wrong reading, nothing in the design catches it, and every one of our degraded documents is of that kind. All results are from one corpus in one language. Both juries are simulated within a single model context per document, each constrained to argue from its own policy file; their independence lies in the separately trained policies, not in separate processes.

References

Wissam Antoun, Fady Baly, and Hazem Hajj. 2020. AraBERT: Transformer-based model for Arabic language understanding. In *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools, with a Shared Task on Offensive Language Detection*, pages 9–15, Marseille, France. European Language Resources Association.

Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. 2020. Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8440–8451, Online. Association for Computational Linguistics.

Mohammed Elkholy, Khalid N. Elmadani, Nizar Habash, and Bashar Alhafni. 2026a. Arabic sentence segmentation across genres and punctuation conditions. *Preprint*, arXiv:2606.08025.

Mohammed Elkholy, Khalid N. Elmadani, Nizar Habash, and Bashar Alhafni. 2026b. Arabic sentence segmentation shared task 2026. In *Proceedings of the Fourth Arabic Natural Language Processing Conference*, Budapest, Hungary. Association for Computational Linguistics.

Ali Farghaly and Khaled Shaalan. 2009. Arabic natural language processing: Challenges and solutions. *ACM Transactions on Asian Language Information Processing (TALIP)*, 8(4):1–22.

Markus Frohmann, Igor Sterner, Ivan Vulić, Benjamin Minixhofer, and Markus Schedl. 2024. Segment any text: A universal approach for robust, efficient and adaptable sentence segmentation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 11908–11941, Miami, Florida, USA. Association for Computational Linguistics.

Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. 2020. Supervised contrastive learning. In *Advances in Neural Information Processing Systems*, volume 33, pages 18661–18673. Curran Associates, Inc.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, pages 46534–46594.

Benjamin Minixhofer, Jonas Pfeiffer, and Ivan Vulić. 2023. Where's the point? self-supervised multilingual punctuation-agnostic sentence segmentation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7215–7235, Toronto, Canada. Association for Computational Linguistics.

Takeru Miyato, Andrew M. Dai, and Ian J. Goodfellow. 2017. Adversarial training methods for semi-supervised text classification. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net.

Jonathon Read, Rebecca Dridan, Stephan Oepen, and Lars Jørgen Solberg. 2012. Sentence boundary detection: A long solved problem? In *Proceedings of COLING 2012: Posters*, pages 985–994, Mumbai, India. The COLING 2012 Organizing Committee.

Ali Safaya, Moutasem Abdullatif, and Deniz Yuret. 2020. KUISAIL at SemEval-2020 task 12: BERT-CNN for offensive speech identification in social media. In *Proceedings of the Fourteenth Workshop on Semantic Evaluation*, pages 2054–2059, Barcelona (online). International Committee for Computational Linguistics.

Sunita Sarawagi and William W. Cohen. 2004. Semi-Markov conditional random fields for information extraction. In *Advances in Neural Information Processing Systems*, volume 17, pages 1185–1192. MIT Press.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652.

Pat Verga, Sebastian Hofstatter, Sophia Althammer, Yixuan Su, Aleksandra Piktus, Arkady Arkhangorodsky, Minjie Xu, Naomi White, and Patrick Lewis. 2024. Replacing judges with juries: Evaluating llm generations with a panel of diverse models. *arXiv preprint arXiv:2404.18796*.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. In *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2024, February 20-27, 2024, Vancouver, Canada*, pages 19632–19642. AAAI Press.

Appendix A Reproducibility

Voter recipes. All five voters: 10 epochs, windows of 180 words with stride 90, maximum sequence length 512, cross-entropy with the boundary class weighted by $\min(\mathrm{neg}/\mathrm{pos}, 8)$, which evaluates to 8.00 on every track. Defaults are learning rate 5×10^{-5} and batch size 16. Differences: `imp_fgm` is AraBERTv0.2-large with FGM $\epsilon = 1.0$; `boost_supcon01` is AraBERTv0.2-large with supervised contrastive weight 0.1; `xlmr-2e5` is XLM-RoBERTa-large at learning rate 2×10^{-5} and batch size 4; `nat_arabertv02_base` is AraBERTv0.2-base and `nat_asafaya_large` is Arabic-BERT-large, both at defaults. Checkpoints are selected on the development split. For Pnx-PA a tuned three-voter subset with $\beta = 0.6$ was used at submission. The four base checkpoints, pinned to the exact Hugging Face revisions our fine-tunes downloaded, are `aubmindlab/bert-base-arabertv02` (revision `016fb9d6`), `aubmindlab/bert-large-arabertv02` (`12c9edc7`), `asafaya/bert-large-arabic` (`a000d4b3`) and `FacebookAI/xlm-roberta-large` (`c23d21b0`). Optimization is AdamW with weight decay 0.01, warmup ratio 0.1 and linear decay, fp16 mixed precision, seed 42, maximum sequence length 512, batch size 16 and learning rate 5×10^{-5} except where a recipe above overrides them; the epoch checkpoint with the best development F_1 is kept. The training entry point and the exact five launch commands are released (`src/train_encoder.py`, `scripts/train_pnx_voters.sh`).

Corpus statistics. Training boundary rate is 0.104 (NoPnx-NP), 0.101 (NoPnx-PA), 0.086 (NP) and 0.083 (PA), over 98,457, 101,784, 124,444 and 128,173 tokens respectively.

Preprocessing and tools. Documents are used exactly as distributed: no normalization of alef or ya forms, ta marbuta, diacritics or tatweel, since the token sequence is the label space and must be preserved. Each encoder uses its own subword tokenizer over the organizer-provided whitespace tokens, and the word-level boundary label is carried by the last subword of each word. No Farasa or CAMeL Tools preprocessing is applied. Training and inference use Python 3.13.3, PyTorch 2.11.0+cu128 and Hugging Face Transformers 4.55.0, as recorded in the released checkpoints; scoring uses the organizers' evaluation script.

Hardware and runs. Encoders were fine-tuned on an NVIDIA RTX 5090 (32 GB), roughly 4 minutes per voter per track; each voter is a single run, and no encoder result is an average. Jury training and adjudication ran through the provider API at maximum reasoning effort, at about six minutes of wall-clock per document per lane, four lanes in parallel. All reported jury results are single runs; run-to-run variability is bounded by the 50-document rerun reported in Limitations.

Metric. We report macro F_1, the per-document mean, computed with the organizers' evaluation script; it is the official metric. Token-pooled micro values are higher because errors concentrate in long documents: on test the ensemble scores 86.23 micro against 84.64 macro on NoPnx-NP, 89.30 against 87.05 on NoPnx-PA, 93.76 against 92.83 on NP, and 95.76 against 94.56 on PA.

Execution environment and replication. The juries execute as agents of the Claude Code framework (public CLI, version 2.1.234 as recorded in the run transcripts), model `claude-opus-5` at maximum reasoning effort. That model string is complete: the Claude 5 family publishes no dated snapshot variants, and the provider's response metadata across our run records names the same identifier for every call. No sampling parameter is set anywhere in the pipeline; temperature, top-p and all other decoding controls remain at provider defaults, so jury decoding is nondeterministic. Each sitting of four documents runs in a fresh context with no memory of any other sitting, two to four sittings in parallel; the framework-free API runner used

for replication pins `max_tokens` to 32,000 with a 30,000-token thinking budget. The release makes the full pipeline repeatable at two levels: every reported number re-derives from released artifacts alone (exact draft rows, all 1,048 doctrine verdicts and all 1,048 zero-shot verdicts, the doctrines, and a strict scorer; the packet builder reproduces our sealed exam packets byte for byte), and the pipeline re-runs end to end from a Makefile (corpus fetch, voter training, probability caching, packet construction), with the jury stage driven either by a single released orchestrator prompt pasted into Claude Code or by a framework-free API runner. The verification script re-derives every number in this paper from source data on each build.

Isolation. Jury agents run under a released file-access guard (a pre-tool-use hook) that denies any path outside their test packets, their own output directory, and their assigned doctrine files (the zero-shot arm assigns none); shell access to the corpus, the gold archive and the repository is denied, and the gold is read in memory by the scoring loop only. Transcript audits of every evaluation run (275 agent transcripts for the zero-shot arm alone) show no access beyond the permitted paths and no gold label vector in any agent context. The agent platform may inject brief session notes into agent context; audits show the only such note that ever appeared named the existence of a training curriculum and carried no corpus content, labels, conventions, or results. The juries' learned files are themselves auditable text: every document id cited and every Arabic 8-gram quoted in any released doctrine occurs in the training split, and none occurs only in the development or test data; every graded training attempt on disk is a training-split document (machine-checked on every build).

Jury prompts. The prompts below are the complete operative texts of the system: everything a user needs to run it is these four blocks plus the released kits. Local file paths and per-jury document lists are replaced by placeholders in angle brackets; the per-track instantiations (format clause, never-clause, grader command, curriculum lists) ship with the code, and a single released orchestrator prompt (`jury/orchestrator_prompt.txt`) runs the full adjudication workflow in Claude Code for any track: sittings of four documents, at most four concurrent agents, per-document verdict banking, resume on crash. All prompts are in English while the documents, the juries' quoted evidence and parts of the doctrines are in Arabic; we did not experiment with Arabic-language prompts. Every historical launcher (all training generations, resume variants, audits, and the voided runs) is additionally released verbatim in `jury/prompts/raw/` with an index, and the full data-lineage audit ships as `CONTAMINATION_MAP.md`.

Training prompt (one per jury; NoPnx-PA instantiation shown). Each jury trains alone: segment a batch of five from scratch, get graded, write the rules behind its errors into its doctrine, carry the doctrine into the next batch. The other tracks differ only in the format clause, the grader command, and the curriculum.

```
You are Jury <j>, a brand-new Arabic sentence-segmentation
judge. You start with zero knowledge of this corpus's
conventions, everything you will ever know comes from your
own graded attempts on TRAINING documents. Work in <repo>.
The ONLY files you may touch: the practice documents listed
below, your own answer files, the grader's mistakes file
named below, and your own doctrine file <doctrine_j>.md.
Touching ANY other file voids the training, answer keys
exist elsewhere in this repository.
FORMAT: Punctuation REMOVED; paragraph structure KEPT as
literal <NL> tokens (each <NL> has its own index). The ¶ on
the token immediately before every <NL> and on the final
token is ALWAYS correct and must never be removed; an <NL>
token itself NEVER carries a boundary. Everything inside
paragraphs is yours to judge. <i> = index anchor, not part
of the text.
YOUR DOCUMENTS, in batches of five, in this order: <87
document ids>
PER BATCH:
1. SEGMENT strictly one document at a time from
<train_docs>/<doc_id>.json. No limit on your reasoning,
first work out what the text IS (genre, register,
structure) and what its natural sentence unit is, then walk
it and argue every uncertain site in writing. Write your
answer to <ans_j>/<doc_id>.json as
{"doc_id":"...","identity":"what this text is","law":"its
sentence law as you read it","reasoning":"your argument at
the hard sites","boundaries":[{"i":<index of the LAST token
of a sentence>,"w":"<that token copied from the
text>","c":"hi|med|lo"}]} the moment it is done, before
opening the next document. Use ONLY properly escaped JSON,
no raw newline characters inside strings.
2. GRADE: run python grade4.py -track nopa -juror <j> -only
<the five ids comma-separated> and read <mistakes_j>.txt in
full.
3. LEARN: for every FALSE CUT and every MISSED, find the
RULE behind it, what do these annotators actually do, and
write it into your doctrine file in your own words, filed
by text type, with your own error as the example. When a
new correction contradicts an earlier entry, fix the
earlier entry and say so. The doctrine is cumulative, never
delete knowledge.
4. Take your updated doctrine into the next batch.
If you hit a usage limit or crash, everything is already
saved; on restart, skip documents whose answer file exists.
When all batches are graded and absorbed, return your
per-batch mean F1 and 4 sentences on your doctrine's core
laws. No web. No PowerShell except the grader command given
above.
```

Adjudication prompt (one per sitting of four documents). The exam prompt: both trained juries in one sitting, arguing every disagreement, unanimity required for any edit.

```
This is exam sitting <n> of <total>. Two TRAINED juries sit together: Jury 0 and Jury 1. Each trained separately on this annotation team's training documents and wrote its own doctrine from its own graded mistakes. You hold both: when Jury 0 speaks, reason strictly from doctrine 0; when Jury 1 speaks, strictly from doctrine 1. They are colleagues with different training histories, not one voice twice.
FIRST read both doctrines ONCE: Jury 0: <doctrine_0> Jury 1: <doctrine_1>
FORMAT: Punctuation REMOVED and paragraph structure KEPT as literal <NL> tokens. The ¶ on the token immediately before every <NL>, and on the final token, is ALWAYS correct and must never be removed; an <NL> token itself NEVER carries a boundary. <i> = index anchor, not part of the text.
YOUR 4 DOCUMENTS, one file each, STRICTLY ONE AT A TIME (finish and record one before opening the next): <four packet paths>
Each document arrives with an automatic system's boundary marks (¶) already drawn in. Treat them as an UNRELIABLE CHEAT SHEET: allowed to consult, never to trust, it makes both kinds of errors. A document whose marks already satisfy its law is FINISHED; unchanged is a common and correct verdict.
PER DOCUMENT, no limit on your reasoning depth:
1. Jury 0 gives its full stance from doctrine 0, what the text is, which register it belongs to, its sentence law, and every place the draft marks are wrong, argued from the text.
2. Jury 1 gives its own independent stance from doctrine 1, site by site.
3. They ARGUE every disagreement. Concede when the colleague's reading matches the annotators' policy better; rebut precisely when it does not. Where a doctrine carries a law earned from a graded mistake in THIS register, that law outranks instinct.
4. RECORD: only changes BOTH juries endorse survive. Unresolved means the draft stands. Write {"doc_id":"...", "add":[{"i":<index>,"w":"<token>"}],"remove":[{"i":<index>, "w":"<token>"}],"argued":"<one line per argued site>"} to <exam_out>/<doc_id>.json IMMEDIATELY on finishing that document. add = token positions that end a sentence but carry no ¶; remove = positions carrying ¶ that do not end a sentence; copy w from the text at the moment you decide. Never add or remove on an <NL> token; never remove a ¶ that sits immediately before an <NL> or on the final token.
RESUME: if <exam_out>/<doc_id>.json already exists, skip that document.
When all are recorded return {"done": <count>, "note": ""}. No web, no PowerShell. Touch NOTHING beyond the two doctrine files, your listed documents, and your own output files, answer keys for this corpus exist elsewhere in this repository and opening anything else voids the evaluation.
```

Per-track format laws. The two prompts above are instantiated per track by substituting one format clause and its matching never-clause:

```
NoPnx-NP: Punctuation REMOVED and paragraph marks REMOVED, plain words only. The final token of a document always ends a sentence. <i> means the next token is token number i (an index anchor, not part of the text). [never-clause: Never remove the ¶ on the final token.]

NoPnx-PA: Punctuation REMOVED; paragraph structure KEPT as literal <NL> tokens (each <NL> has its own index). The ¶ on the token immediately before every <NL> and on the final token is ALWAYS correct and must never be removed; an <NL> token itself NEVER carries a boundary. Everything inside paragraphs is yours to judge. <i> = index anchor, not part of the text. [never-clause: Never add or remove on an <NL> token; never remove a ¶ that sits immediately before an <NL> or on the final token.]

Pnx-NP: Punctuation KEPT. Punctuation marks are ordinary tokens with their own indices, and a sentence usually ends ON its punctuation mark when one is present. There are NO <NL> tokens in this track. The final token always ends a sentence. <i> is an index anchor marking that the next token is token number i; it is not part of the text. [never-clause: Never remove the ¶ on the final token.]

Pnx-PA: Punctuation KEPT (marks are ordinary tokens, and a sentence usually ends ON its mark) AND paragraph structure KEPT as literal <NL> tokens. The ¶ on the token immediately before every <NL>, and on the final token, is ALWAYS correct and must never be removed; an <NL> token itself NEVER carries a boundary. <i> = index anchor, not part of the text. [never-clause: Never add or remove on an <NL> token; never remove a ¶ that sits immediately before an <NL> or on the final token.]
```

Zero-shot control prompt. Identical sitting structure, no doctrines, juries told they are untrained; verdicts write to a separate directory:

```
This is exam sitting <n> of <total>. Two juries sit together: Jury 0 and Jury 1. Neither has been trained on this corpus and neither holds any written policy or examples from it; each reasons only from its own knowledge of Arabic and from the text in front of it. You hold both: when Jury 0 speaks, reason as Jury 0; when Jury 1 speaks, reason independently as Jury 1. They are colleagues who reach their own readings, not one voice twice.
FORMAT: <the track's format clause, character-identical to the doctrine run's>
YOUR 4 DOCUMENTS, one file each, STRICTLY ONE AT A TIME (finish and record one before opening the next): <four packet paths>
Each document arrives with an automatic system's boundary marks (¶) already drawn in. Treat them as an UNRELIABLE CHEAT SHEET: allowed to consult, never to trust, it makes both kinds of errors. A document whose marks already satisfy its law is FINISHED; unchanged is a common and correct verdict.
PER DOCUMENT, no limit on your reasoning depth:
1. Jury 0 gives its full stance, what the text is, which register it belongs to, its sentence law, and every place the draft marks are wrong, argued from the text.
2. Jury 1 gives its own independent stance, site by site.
3. They ARGUE every disagreement. Concede when the colleague's reading is better supported by the text; rebut precisely when it is not.
4. RECORD: only changes BOTH juries endorse survive. Unresolved means the draft stands. Write {"doc_id":"...", "add":[{"i":<index>,"w":"<token>"}],"remove":[{"i":<index>, "w":"<token>"}],"argued":"<one line per argued site>"} to <exam_out_zeroshot>/<doc_id>.json IMMEDIATELY on finishing that document. add = token positions that end a sentence but carry no ¶; remove = positions carrying ¶ that do not end a sentence; copy w from the text at the moment you decide. <the track's never-clause>
RESUME: if <exam_out_zeroshot>/<doc_id>.json already exists, skip that document.
When all are recorded return {"done": <count>, "note": ""}. No web, no PowerShell. Touch NOTHING beyond your listed documents and your own output files, answer keys for this corpus exist elsewhere in this repository and opening anything else voids the evaluation.
```

Appendix B Error analysis

All numbers in this section are recomputed from the cached voter probabilities and the released gold by our verification script; the setting is the NoPnx-NP track with each voter and the ensemble mean thresholded at 0.5. On the development split the ensemble makes 3,749 errors, and 1,811 of them (48.3%) are sites where all five voters are on the wrong side of the threshold: the models agree with each other and disagree with the annotator. At the remaining sites the voters split, but no voter is worth promoting, since every voter alone scores below the ensemble (solo macro F_1 from 80.74 to 84.56 against the ensemble's 85.62), so the dissenting vote at a mixed site is not a signal a fixed combination rule can exploit. The confident half cannot be thresholded away either. The median ensemble

probability at a false positive site is 0.86, while 22.7% of true boundaries fall below that value: a global threshold high enough to remove the typical false positive forfeits nearly a quarter of the real boundaries, and calibration rescales these probabilities monotonically and therefore cannot reorder them. Nor are the errors spread evenly; the worst fifth of development documents holds 51.4% of all error sites, so failure concentrates in particular documents, which is what a method that conditions on the document, rather than sharpening a shared probability model, is positioned to fix. The test split shows the juries doing exactly that. Of the ensemble's 3,413 error sites, the juries corrected 1,110 and introduced 230 new errors. 299 of the corrected sites (26.9%) are sites where every voter had been wrong, a class that no reweighting, recalibration, or voter promotion can reach, because no voter disagrees with the mistake; the remainder are mixed sites where the juries sided with the minority reading when the document's convention licensed it. The zero-shot arm reaches fewer errors of every kind on the same split: it corrected 917 sites while introducing 187, and 227 of its corrections (24.8%) were all-five-wrong sites against the doctrine's 299 (26.9%), so the learned policy both edits more and aims slightly better.

Appendix C The juries' reasoning while learning

The policy files are the training record: every entry carries the batch that produced it, and superseded rules are struck through rather than deleted. Batch numbers in the quoted files count grading rounds, which did not always hold five documents: Jury 0's first two rounds graded 18 and 23 documents and Jury 1's first round 19. Figure 4 reproduces one abridged entry; below it we give the central revision arc of the NoPnx-PA Jury 0 doctrine and two entries from Jury 1, verbatim except that Arabic examples are transliterated or elided and long passages are abridged. One reading note is essential: NoPnx-PA input carries no punctuation at all (the training split holds 101,784 tokens and not one punctuation mark), so when the jury below speaks of commas and full stops it is reasoning about the marks the source text must have carried before they were stripped, inferred from where the annotators' boundaries fall. That inference is the point: the gold is the source's punctuation with the marks deleted, and the jury is reconstructing the deleted convention.

Complete doctrines. All six policy files are provided verbatim as supplementary material (UTF-8 Markdown with the Arabic quotations, plus a README) and in the release under `jury/doctrines/`: the NoPnx-NP pair (folder `NoPnx-NP`), the post-deadline NoPnx-PA retrain of Table 1 (`NoPnx-PA`), and the shared punctuated pair (`Pnx-NP-PA`). Two of these pairs were trained on punctuation-free text and must not be confused. The NoPnx-NP pair, which produced the 89.1 leaderboard score, trained on the curated 39-document curriculum. The punctuated pair also trained on NoPnx-NP-format text, but across most of the training split (its files record 141 and 123 graded documents and describe themselves as NoPnx-NP doctrines); it was never used on the NoPnx-NP track and adjudicated Pnx-NP and Pnx-PA, behind the 92.0 and 95.30. Every excerpt in this appendix appears verbatim in those files.

Why the roles are split this way. The data-lineage audit of Appendix A voided the earlier NoPnx-NP training runs, so the NoPnx-NP pair was rebuilt from scratch in the final hours before the deadline, and usage limits allowed a curated curriculum of 39 documents rather than the full split. The small curriculum is not a weakness of the design but a constraint of the clock: Table 1 shows this pair still adds +4.85 macro-F_1 over the ensemble, +1.89 of it attributable to the learned policy beyond the zero-shot scaffolding, and Appendix B shows that its corrections reach error sites no reweighting of the voters can. The whole-corpus pair was kept for the punctuated tracks because a policy learned on punctuation-free text transfers upward: punctuation adds surface cues without changing the annotators' conventions, the pair's doctrine carries format-specific law sections for punctuated input (Limitations), and the restraint it learned is exactly what the punctuated tracks reward, where untrained juries lose ground (Section 5).

Jury 0, batch 2: diagnosing its own first rule.

> "LAW 0, REVISED (batch 2). This is the most important correction in the file. Batch 1's LAW 0 was wrong in its central claim and I acted on it, which is why batch 2 inverted my error profile from 4.8:1 under-cutting to 22:1 over-cutting. I did not learn a rule in batch 1; I learned a base rate, and a base rate is not transferable. [The superseded batch-1 rule follows, struck through.] The two errors in that paragraph: (1) the comma is NOT a boundary. This one clause is responsible for something like 300 of my 354 false cuts in batch 2: a comma-separated list, an appositive, a parenthetical gloss, a circumstantial clause, all of these were commas in the source [editorial note: the input was fully stripped of punctuation; the jury inferred these latent commas from the gold boundaries] and all of them weld. (2) '6 to 14 tokens' was a property of the batch-1 sample, not of the corpus."

LAW 5, batch 2 (abridged, verbatim). "Off-by-one-clause remains a real, separate failure mode. In batch 2 nearly every one of my 16 misses was paired with a false cut 1–2 clauses away: I had the right idea and the wrong site. **REVISION to the batch-1 tactic.** Batch 1 said the annotators cut at the *first* available *wa/fa* in a run, not the second. Batch 2 shows the opposite as often as not, so the first-joint heuristic is retired."

Figure 4: A policy update written by a jury after its second graded batch, retiring a rule it had induced from the first. Each entry cites the document and token offsets of the errors that forced the change.

Jury 0, batch 13: catching the over-correction.

"LAW 0′′, REVISED AGAIN (batch 13). Batches 11, 12 and 13 falsified the WELD default as hard as batch 2 falsified the CUT default. Precision ran 96 to 100% while recall ran 57 to 77%: about 50 misses against about 8 false cuts, three batches running. 'Default weld' is the single most expensive sentence I have ever written in this file. The corrected posture is neither 'cut' nor 'weld' but a protected-list procedure: CUT at every clause boundary where a fresh finite predication begins after a complete preceding predication, UNLESS the site is on the protected list. This is not a return to batch 1. Batch 1 cut at connectives; this cuts at predications, and it carries an explicit exemption list built from the 354 false cuts of batch 2. The connective (*wa*, *fa*, *thumma*, *lākin*) is neither evidence for nor against: what matters is whether a new finite clause with its own predicate starts there, and whether the site is protected."

Jury 1, batch 2: refusing to double-count evidence. A scheduling fault re-served Jury 1 its first batch. The jury noticed and wrote:

"[This is] the same grading output as batch 1. Same 19 doc IDs, same F1s to one decimal, and the same token offsets and quoted spans: I checked about 30 items against the citations already written into this file, and every one matches character for character. So batch 2 contains no new evidence. I am recording that rather than pretending to a second sample. I used the re-read the only way it can honestly be used: as a full audit of batch 1's doctrine against the complete mistake list. The audit found four entries that are wrong or misfiled and about a dozen mistakes I never filed at all."

Jury 1, batch 5: separating a right behaviour from a wrong rule.

"K1 IS WRONG [REVISED batch 5]. Batch 1 wrote 'each hemistich is a unit' from two MISSED items in one poem. Reading that document's layout now: the poem is printed one hemistich per line, so the hemistich boundary was already being handed to me by the line break. Gold cut inside a verse line only twice in about 25 lines. My batch-1 behaviour, treat the line as the unit, was right, and the rule I generalized from it was not a rule about hemistichs at all."

Four behaviours matter here, and none was scripted: the jury distinguishes a base rate from a rule, prices its own worst sentence, declines to treat duplicated evidence as confirmation, and separates a behaviour that happened to be right from the wrong generalization it had written down for it. This is the mechanism whose value the ablation in Table 1 measures. Figure 3 shows the same record quantitatively for the NoPnx-PA juries. Every training attempt is made blind, on a document the jury has never seen, before that batch's gold is revealed, so the running score is in effect a rolling validation curve; averaged over a moving window of ten batches, Jury 0 rises from 85.1 to 90.9 and Jury 1 from 88.5 to 94.5, each ending at its maximum, and the curves rise because the files grow.

Appendix D How the juries work a document

Two complete adjudication records in the juries' own words as recorded at decision time, abridged where marked, from the rerun of the Limitations section (identical doctrines and protocol to the submitted system; that run's records are in transliteration, which this template can typeset). Both documents are wins: the first rose from 28.6 to 100.0 F_1, the second from 77.1 to 87.5.

A 20-token sura (Surat an-Nas, Q114).

"REGISTER: the whole document is Surat an-Nas (Q114) entire, 20 tokens, standing alone, not a quotation inside prose. J0 S21: for the Quran the RECORD is the AYA, every *wa/fa* rule suspended, and the text carries its own hard external test (known verse count plus known fasila list). J1 S16/S12c: verse cuts at the rhyme and nowhere else. Both juries: 6 ayat = 6 units. J1's objection from S2b (quoted verses are ONE unit) was withdrawn: that datapoint was a Quranic fragment embedded in Ibn Tufayl prose, and the material J1 called verse 1 and verse 2 is in fact two clauses of a single aya (Q8:17), so it is no evidence against per-aya segmentation of a standalone sura. J0 S21 is evidence from THIS register and outranks it. [Five adds follow, each the end of an aya, each endorsed by both.] @10 al-waswas ARGUED AND REJECTED: J1's mechanical rhyme scan (cut at every token carrying -as) fires here. J0 rebutted with S21's sharpest lesson: a fasila can repeat INSIDE its own verse, so take the LAST occurrence before the next ending, not the first (the an-Nahl trap that cost J0 a double penalty). al-waswas is verse-internal; cutting it would give 7 units for a 6-aya sura. J1 conceded. NO CUT. DENSITY: 20/6 = 3.3 tokens per unit, below J1's verse band of 4 to 6, but an exact external invariant (the sura has exactly 6 ayat) outranks a density heuristic; J0 S24.2 orders verification against the invariant rather than the ear."

A 746-token translated novel chapter (Doctorow, *Little Brother*).

"REGISTER: Hindawi Arabic translation of Cory Doctorow, Little Brother, chapter 1: first-person YA novel, running prose, not layout. J1 bucket (d) translated prose, cut at the source-language period, 13 to 17 tokens; J0 S8 translated literary fiction about 14. [...] @188: J1 wanted the mark removed (English has a comma before the but), J0 S1.3 holds that *lakin* opens a sentence. UNRESOLVED, draft stands. @205: the 36-token unit is the worst density outlier in the document. J1 first objected (parallel negations do not cut), then conceded that this is an affirmative-then-negative contrast, not a parallel negation, and that bucket (d) points at the source period; J0: *wa* plus finite verb in narrative. ADD. [...] @624 to @647, THE ONE STRUCTURAL ERROR IN THIS DOCUMENT: the draft read four data lines as display-line records and split them off, leaving a unit that ends on a dangling *kana* with no predicate and a next unit opening on a bare predicate with no subject. Correct parse: the quote-plus-tag sentence ends at @624, and the identity block is a dash parenthetical between *kana*'s subject and its predicate, one sentence. J0's false-heading trap (does the verb after the candidate agree with it and lack any other subject) and J1's rule (parse the verb's subject before deciding a heading's extent) are the same law, and both fire. ADD 624; REMOVE 627, 633, 638, 644, 647. @667 and @708: English joins these to the following clause, but the Arabic translator plausibly broke here; no jury evidence strong enough either way. NO CHANGE."

Six behaviours are visible across the two traces and none was scripted: register identification before any edit; suspension of general rules when a register-specific law applies; verification against an external invariant in place of intuition; an objection withdrawn after the jury re-audited its own evidence; disagreements left unresolved with the draft standing; and a structural correction argued from syntax rather than surface pattern.

Raw internal reasoning (abridged; Arabic transliterated). The platform records the model's internal reasoning; the excerpt below, from a hadith document in the same rerun, shows the full pattern of a single decision: a candidate edit proposed, endorsed by two laws, rejected against a third, relocated one token, and verified by density. It corroborates that the recorded arguments above reflect the actual process rather than post-hoc summaries.

> "I am looking at this hadith document from Sahih al-Bukhari, checking my current mark placements against the isnad-breaking doctrine: the chain of narration splits at each *qala*, since these are separate transmission records rather than one continuous sentence. Jury 1's notes add nuance: the final *qala* still welds to the quoted speech regardless of isnad length. [...] Now for *faqala* at token 28: this opens a new speech act. J0 §22 names *faqala* a boundary head in hadith, and J1's §9d and §18 both support cutting before a narrative *faqala*, so a cut looks endorsed by both juries. But J0's gold example has the boundary after a complete narrative clause, whereas here it falls after a quoted imperative, and per §9c an opening imperative right after the introducer welds to it rather than breaking away; so '*qala lahu fi hijjat al-wada' istansit al-nas*' stays one unit and that cut should not happen. Both juries converge on adding the boundary at token 27 (*al-nas*) instead, once the next full clause ('*faqala la tarji'u...*') begins. Checking density: 37 tokens and 3 boundaries is about 12.3 tokens per unit against J0's hadith benchmark of roughly 9.3; adding this one boundary brings it to 9.25."